Document technique

Projet Nudge

Logiciel lourd de suivi de projet avec relance automatique par mail

NUDGE

PILOTEZ VOS PROJETS. ZERO OUBLI , ZERO RETARD

Auteurs du document	
Nom	**Prénom**
Gironcel	Jason
Grimoult	Hugo
Landry	Antoine
Mussard	Lucas
Morin	Ného

Date	**Version**	**Modifications**	**Auteur**
31/03/2026	V 1.0	Toute les pages (1-14)	Jason, Antoine, Hugo
08/04/2026	V 1.1	Toute les pages	Lucas, Ného

Sommaire

1. Présentation du projet **4**
Valeur ajoutée métier 4
2. Équipe et répartition des rôles **4**
Mode de fonctionnement retenu 5
3. Cahier des charges synthétique et périmètre retenu **5**
Fonctionnalités minimales retenues pour le MVP 5
Périmètre recommande intégré au document 5
Hors périmètre 5
4. Choix techniques et environnement de développement **5**
5. Architecture de la solution **6**
Description des flux principaux 6
Règles de relance 6
6. Modélisation des données **7**
Relations fonctionnelles 7
Diagramme UML 8
7. Organisation du projet : retro planning, jalons et réunions **8**
Jalons du projet 9
Planning prévisionnel des réunions / videos 9
8. Suivi d'avancement **10**
Indicateurs de suivi proposes 10
9. Gestion des risques **10**
10. Stratégie de tests **11**
Scénarios de validation prioritaires 11
11. Documentation d'installation et d'utilisation **12**
11.1 Installation 12
11.2 Utilisation 12
11.3 Pré requis pour la démonstration 12
12. Difficultés rencontrées et solutions retenues **12**
13. Conclusion **13**

1. Présentation du projet

Nom du projet : Nudge

Objectif projet :

Création d'un logiciel de bureau (lourds), pour :

- Centraliser le suivi des projets
- Piloter les tâches
- Identifier les échéances
- Automatiser les relances par mail

Logiciel installé localement sur un poste utilisateur

Valeur ajoutée métier

Centralisation des informations projet, tâches, responsables et historiques de relance.

Lecture rapide de la situation grâce à un tableau de bord synthèse.

Réduction des oublis grâce aux alertes automatiques.

Meilleure traçabilité des actions réalisées.

Base saine pour faire évoluer le prototype vers un outil plus complet.

2. Équipe et répartition des rôles

L'équipe est composée de cinq membres.
Présentation des rôles et des membre associés :

Membre	Rôle principal	Responsabilites detaillees
Jason Gironcel	Responsable technique	Validation des choix techniques, structuration du dépôt, supervision de l'architecture logicielle, vérification de la cohérence globale.
Hugo Grimoult	Developpeur Full Stack	Développement de l'interface, logique métier, liaison avec la base de données, intégration des règles de relance.
Landry Antoine	Chef de projet	Planification, organisation des réunions, suivi du rétroplanning, préparation des visions, centralisation des décisions.
Lucas Mussard	Directeur artistique	Maquettes, ergonomie, identité visuelle, cohérence de l'expérience utilisateur, supports de présentation.
Ného Morin	Testeur, Soutien	Tester le programme final et aider les autres membres du projet.

Mode de fonctionnement retenu

Validation hebdomadaire des objectifs de la semaine.

Suivi des tâches dans un tableau de projet partagé.

Travail par version courte pour disposer rapidement d'un MVP démonstrable.

Documentation continue afin d'éviter une rédaction de dernière minute.

3. Cahier des charges synthétique et périmètre retenu

Fonctionnalités minimales retenues pour le MVP

Création, modification et consultation de projets.

Création, modification et suppression de tâches.

Association d'une tâche à un projet et à un responsable.

Gestion des dates d'échéance, priorités et statuts.

Tableau de bord global.

Mécanisme de relance automatique par mail.

Historique des relances.

Interface graphique claire et exploitable.

Périmètre recommande intégré au document

Filtre des tâches en retard ou proches de l'échéance.

Visualisation de l'avancement global par projet.

Priorisation des tâches.

Mode simulation pour l'envoi des mails lors des démonstrations.

Hors périmètre

Application collaborative temps réel.

Authentification d'entreprise complexe.

Moteur de workflow avancé.

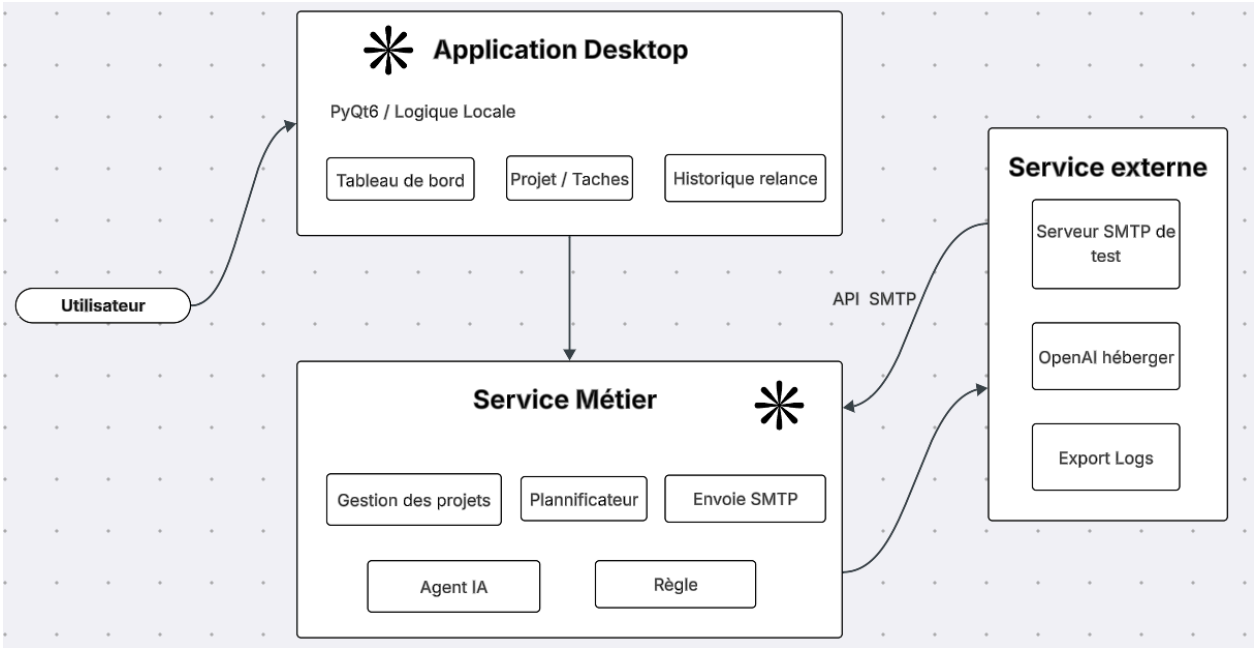
4. Choix techniques et environnement de développement

Composant	Choix retenu	Justification
Langage	Python 3.12	Rapide à développer, lisible, riche écosystème et adapté à un prototype de bureau multi plateforme.

Interface graphique	PyQt 6	IHM plus moderne et plus flexible que Tkinter pour un tableau de bord structure et une application présentable.
Base de données	SQLite 3.53.0	Base locale légère, sans serveur, idéale pour un logiciel lourd étudiant et simple à embarquer.
Relances automatiques	APScheduler 3.11.2	Permet de vérifier régulièrement les échéances et de déclencher les règles de relance.
Mail	smtplib + email (bibliothèque python)	Solution simple, standard et suffisante pour une démonstration ; simulation possible si besoin.
IA	OpenClaw 2026.4.11	Permet de reformuler des mails ou de synthétiser des informations sans bloquer le cœur de l'application.
Gestion de version	Git + GitHub	Historique des versions, travail d'équipe et traçabilité des contributions.
Gestion de projet	GitHub Projects	Suivi visuel des tâches, des jalons et de l'avancement.

5. Architecture de la solution

L'architecture retenue suit une logique en couches afin de séparer clairement l'interface, la logique métier, la persistance des données et les services externes. Cette organisation facilite les tests, la maintenance et l'évolution du prototype.



Situation détectée	Règle	Action

Tâche proche de l'échéance	J-2	Envoi d'un rappel au responsable.
Tâche à échéance le jour même	J	Relance le matin avec résumé de la tâche.
Tâche sèche non terminée	J+1 et ensuite quotidien	Relance automatique tant que le statut n'est pas terminé.
Tâche bloquée sur une longue durée	hebdomadaire	Message de suivi avec demande de mise à jour.

6. Modélisation des données

La base de données est centrée sur quatre entités principales : Projet, Tâche, Utilisateur et Relance. Ce noyau couvre le besoin minimal tout en restant évolutif. Le choix de SQLite permet de conserver une structure relationnelle claire sans ajouter de serveur de base de données.

Relations fonctionnelles

Un projet possède plusieurs tâches.

Une tâche appartient à un seul projet.

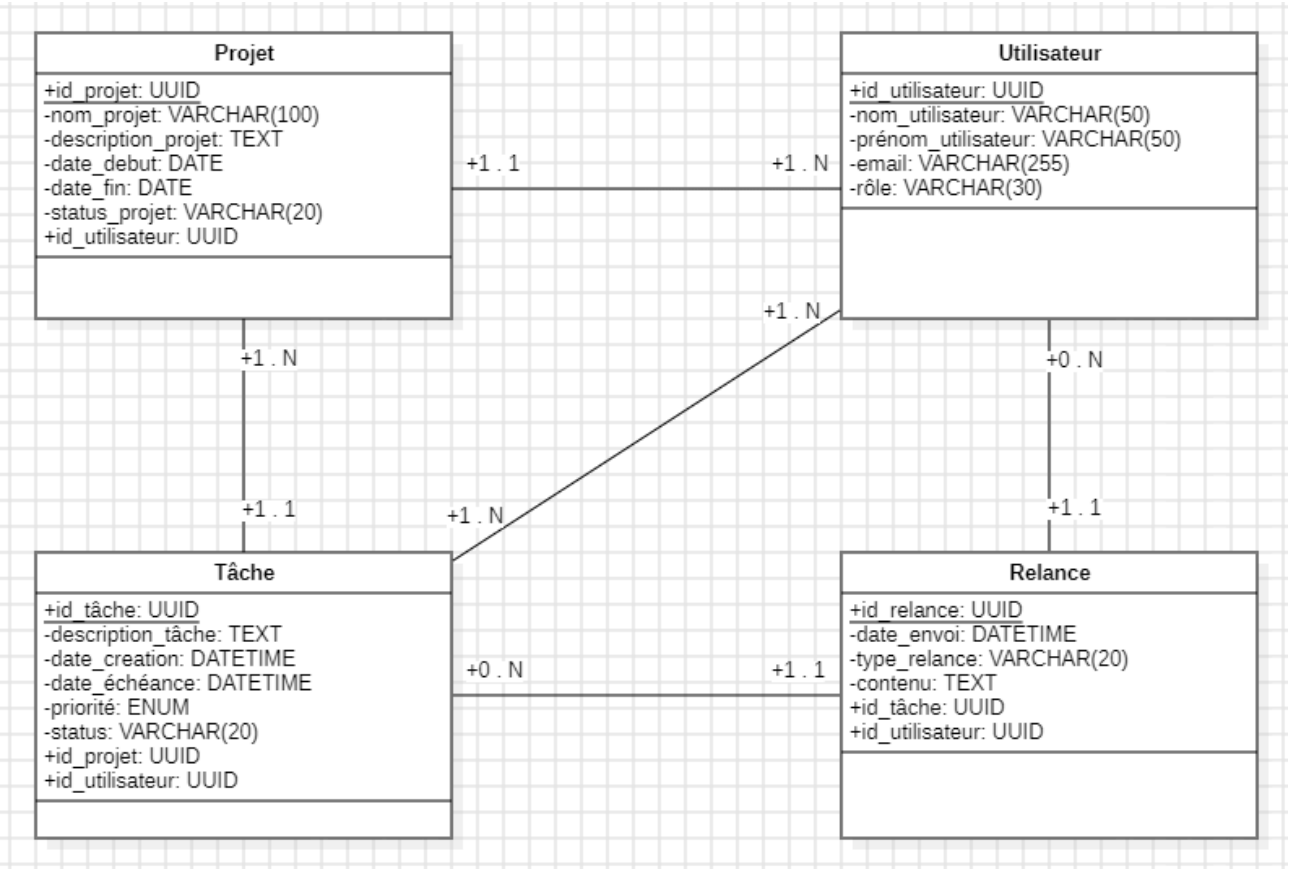
Une tâche est associée à un responsable.

Une tâche peut produire plusieurs relances.

Une relance référence le destinataire concerné et la tâche associée.

Entité	Attributs essentiels
Projet	id_projet, nom, description_projet, date_debut, date_fin, statut_projet, id_utilisateur
Tâche	id_tâche, nom_tâche, description_tâche, date_creation, date_échéance, priorité, statut, id_projet, id_utilisateur
Utilisateur	id_utilisateur, nom_utilisateur, prénom_utilisateur, email, rôle
Relance	id_relance, date_envoi, type_ relance, contenu, id_tâche, id_utilisateur

Diagramme UML



7. Organisation du projet : retro planning, jalons et réunions

Période	Livrable / phase	Contenu principal	Etat au 31/03
Mi-mars 2026	Analyse du besoin	Lecture du sujet, cadrage, répartition initiale des rôles, identification du MVP.	Fait
19/03 -> 01/04	Visio 1	Présentation de l'avancement, validation du besoin, ajustement du planning.	Fait
Avant 03/04 16h	PowerPoint commercial	Contexte, solution, maquette, valeur ajoutée, architecture globale.	Fait
04/04 -> 10/04	Visio 2	Vérification du prototype, discussion des blocages, planification.	Fait

Avant 13/04 12h	Document technique	Organisation, architecture, modélisation, tests, installation.	Fait
Jusqu'au 30/04	Prototype fonctionnel	IHM, BDD, règles de relance, historique, démonstration poste à poste.	À développer
Mai 2026	Stabilisation	Corrections, ergonomie, répétition de la demonstration, support oral.	A venir
03/06/2026	Oral final du groupe 6	Présentation de 30 minutes maximum devant le client.	A venir

Jalons du projet

Jalon 1 : besoin analyse et périmètre MVP valide.

Jalon 2 : maquette et choix techniques valides.

Jalon 3 : socle fonctionnel projets + tâches + utilisateurs opérationnel.

Jalon 4 : relances automatiques actives.

Jalon 5 : prototype complet et démontrable.

Jalon 6 : soutenance préparée et testée.

Planning prévisionnel des réunions / videos

Type	Fréquence / date	Objectif	Participants
Point équipe	Chaque lundi 18h00 - 18h30	Suivi des tâches, blocages, décisions de la semaine.	Toute l'équipe
Point technique	Chaque jeudi 18h00 - 19h00	Revue du code, schema BDD, architecture, tests.	Responsable technique + développeurs
Visio 1 encadrant	Entre le 19/03 et le 01/04	Présenter l'avancement initial et faire valider la direction prise.	Equipe + enseignant
Visio 2 encadrant	Entre le 04/04 et le 10/04	Montrer le prototype et ajuster la priorisation.	Equipe + enseignant
Répétition demo	Semaine du 27/04	Tester le déroulé de la démonstration individuelle.	Toute l'équipe
Répétition orale finale	Semaine du 25/05	Synchroniser les prises de parole et la démonstration.	Toute l'équipe

8. Suivi d’avancement

Tâches réalisées	Tâches en cours	Tâches restantes
Analyse du besoin; Repartition des roles; Préparation du diaporama; Définition du concept Nudge; Première maquette de l'interface; Choix d'une architecture générale; Modélisation BDD; Rédaction du document technique; Préparation de la visio;	Structuration du dépôt Git; Définition fine des règles de relance; Développement du prototype complet; Intégration SQLite;	Planificateur automatisé; Envoi SMTP / simulation; Historique des relances; Tests et packaging; Préparation de la demi-finale;

Indicateurs de suivi proposes

Taux de fonctionnalités MVP terminées.

Nombre de bugs ouverts / corrigés.

Stabilité des scénarios de démonstration.

Respect des dates de remise .

État des tâches dans l' outil de gestion de projet.

9. Gestion des risques

Risque	Impact	Probabilité	Mesures préventives / correctives
Périmètre trop ambitieux	Elevée	Moyenne	Prioriser un MVP clair, geler les fonctionnalités secondaires et valider rapidement le cœur du besoin.
Retard sur l’interface graphique	Moyen à élevé	Moyenne	Utiliser une maquette simple, privilégier des composants standards et iterer progressivement.
Dysfonctionnement ou blocage du service d'envoi de mails	Elevée	Faible à Moyenne	Configuration correcte des enregistrements DNS (SPF/DKIM); utiliser un service tiers réputé au lieu d'un serveur SMTP local.

Mauvaise gestion des dates / échéances	Elevée	Moyenne	Centraliser les calculs de date dans un module dédié et tester les cas limites.
Dépendance trop forte à l' IA	Moyen	Faible	Rendre l'agent IA optionnel et garder un mode manuel complètement fonctionnel.
Problèmes de coordination d'équipe	Moyen	Faible à moyenne	Mettre en place des points hebdomadaires courts et une traçabilité des décisions.

10. Stratégie de tests

Type de test	Liste exhaustive	Critère de succès
Tests unitaires	Création d'une tâche, changement de statut, calcul du retard, vérification de la priorité.	La logique métier renvoie les résultats attendus sur des cas normaux et limites.
Tests d'intégration	Enregistrement en base, récupération des projets, génération d'une relance, écriture dans l'historique.	Les composants communiquent correctement entre eux.
Tests IHM (Interface Homme Machine)	Ajout / modification / suppression via l'interface.	L'utilisateur peut réaliser les actions sans blocage ni incohérence.
Tests mail	Génération du contenu, envoi simple, envoi SMTP de test.	Le contenu est correct et la relance est tracée.
Tests de non régression	Rejouer les scénarios principaux après correction.	Aucune fonctionnalité déjà validée ne doit se dégrader.
Recette de démonstration	Scénario complet du projet à la relance automatique.	La démonstration s'exécute sans erreur visible.

Scénarios de validation prioritaires

1. Créer un projet puis y ajouter plusieurs tâches.
2. Associer un responsable avec une adresse mail.
3. Définir une échéance proche et vérifier la détection J-2.
4. Faire passer une tâche en retard et observer la relance.
5. Consulter l'historique des relances.
6. Modifier le statut en Terminale et vérifier l'arrêt des relances.

11. Documentation d’installation et d’utilisation

11.1 Installation

1. Installer Python 3.12.
2. Récupérer le dépôt Git du projet.
3. Créer un environnement virtuel Python.
4. Installer les dépendances du fichier “requirements.txt”.
5. Configurer les paramètres de mail dans un fichier “.env”.
6. Lancer l'application via le script principal “main.py”.

11.2 Utilisation

1. Ouvrir l'application.
2. Créer un projet avec son nom, sa description et sa date de fin prévisionnelle.
3. Ajouter les responsables avec leur adresse mail.
4. Créer les tâches, choisir leur priorité et leur échéance.
5. Consulter le tableau de bord pour visualiser l'état global.
6. Vérifier les relances générées et l'historique associé.
7. Mettre à jour le statut des tâches jusqu'à leur clôture.

11.3 Pré requis pour la démonstration

Base de données initialisée.

Jeu de données de test avec plusieurs projets et tâches.

Mode simulation mail disponible en cas de panne SMTP.

Connexion réseau seulement si l'agent IA OpenClaw est active.

12. Difficultés rencontrées et solutions retenues

Difficulté	Solution retenue
Concilier ambition fonctionnelle et delai court	Adopter un MVP puis ajouter les améliorations progressivement.
Garantir une démonstration fiable des mails	Prévoir un mode simulation et un SMTP de test.
Concevoir une IHM présentable sans trop de complexité	Choisir PyQt 6 et réutiliser des composants standards.
Éviter que l'IA devient un point de blocage	Rendre OpenClaw externe et facultatif.
Garder une application portable	Utiliser Python, SQLite et une configuration simple sous Windows et Linux.

13. Conclusion

Nudge répond à un besoin concret de suivi de projet : centraliser les informations, visualiser les échéances, réduire les oublis et automatiser les relances.

Le présent document propose une version technique complète, cohérente avec le diaporama fourni et avec le cahier des charges. Il formalise une organisation d'équipe, un choix de stacks, une architecture, un schéma de données, un plan de tests et un cadre d'installation. Il constitue ainsi une base solide pour la remise du document technique et pour la suite du développement du prototype.